



School of Computer Science

COMP 8347 “Internet Applications and Distributed
Systems”

Summer 2026

Week 3

Django Basics: Models, Views, Templates; Databases & ORM

DR. OLENA SYROTKINA



University of Windsor

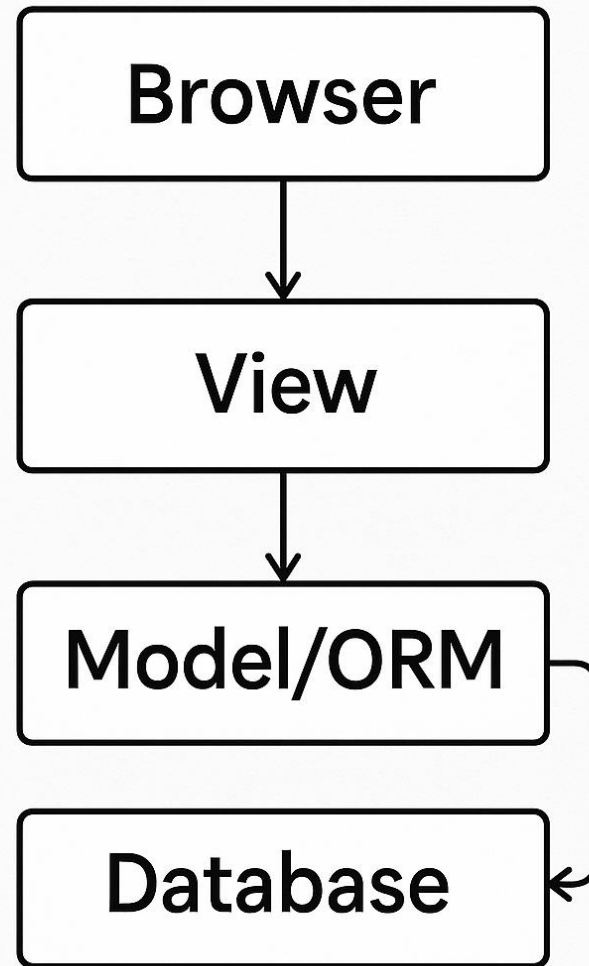
Copyright Notice

This content is protected and may not be shared, uploaded, or distributed. Any unauthorized distribution, reproduction or sharing of this material, including uploading to **CourseHero**, **Chegg**, **StuDocu** or other websites, is **strictly prohibited** and may be subject to legal action. Students are granted access to this material solely for their personal use and may not use it for any other purpose without the express written consent of the instructor. Thank you for respecting our intellectual property rights.

Agenda

- Introduction
- Models (data definitions)
- Views (the logic that ties it together)
- Templates (how we present data to the user)
- Your Second Django App

Introduction



Browser → View → Model/ORM → Database → back

1. Browser (Client)

The user's web browser sends an HTTP request (GET, POST, etc.) to your Django server. E.g. A user visits

<https://example.com/products/>.

2. View

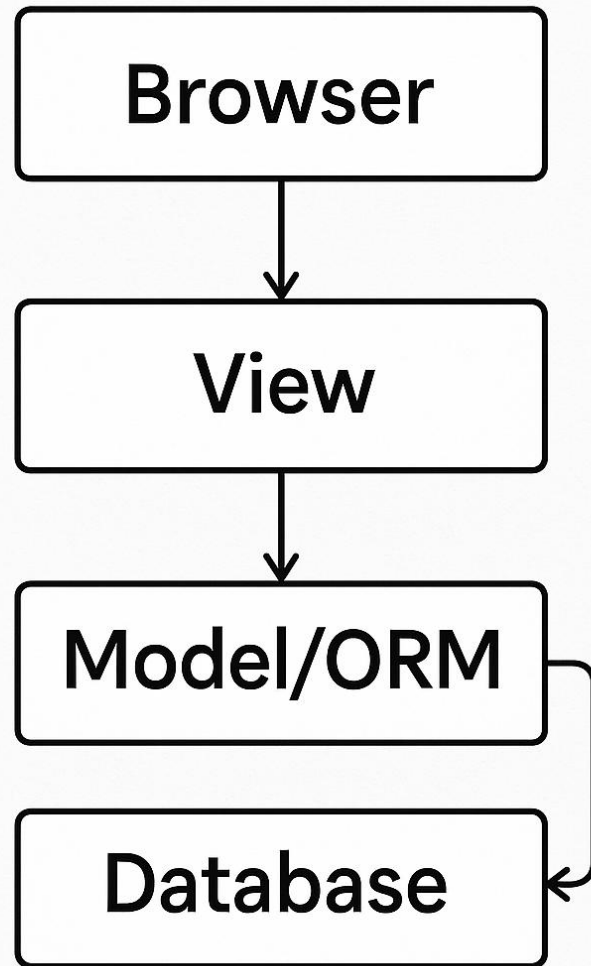
Django View Function/Class receives the request object from the browser and acts as the “controller” in MVC terms.

Tasks:

- Parses URL parameters and form data.
- Decides which data is needed.
- Calls the appropriate Model or performs business logic.
- Chooses which Template to render (even though template isn't shown in this simplified flow).

E.g. `products/views.py` function queries the database for product list.

Introduction



Browser → View → Model/ORM → Database → back

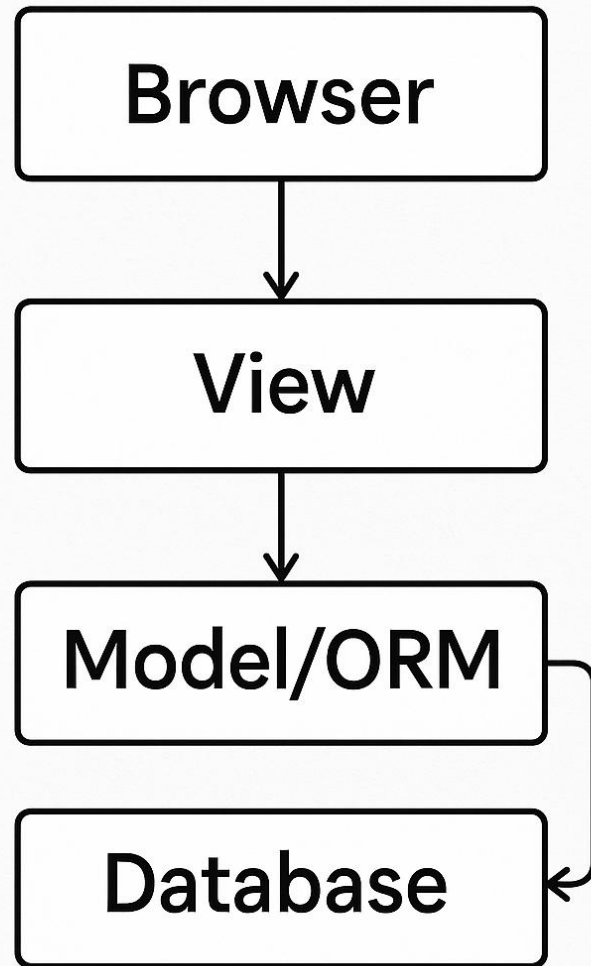
3. Model / ORM

- **Django Models:** Python classes describing database tables and relationships.
- **Django ORM:** The Object-Relational Mapper that converts Python method calls into SQL.
- **Process:**
 - ✓ View calls `Product.objects.all()`.
 - ✓ ORM generates SQL e.g. `SELECT * FROM products_product;`
 - ✓ Database returns rows as Python objects.

4. Database

- **Role:** Stores persistent data (e.g., PostgreSQL, MySQL, SQLite).
- Handles queries generated by the ORM and returns results.

Introduction



5. Return Path (Back to Browser)

- Data flows **backward**:
 1. Database → ORM → View.
 2. View injects this data into an HTML Template.
 3. Django renders the template into an HTTP response.
 4. Response is sent to the **Browser**, where the user sees the final web page.

Browser → View → Model/ORM → Database → back

Django Models: Defining Data

Live Demo: Create an App for Data

```
python manage.py startapp classroom
```

Add the App to Settings

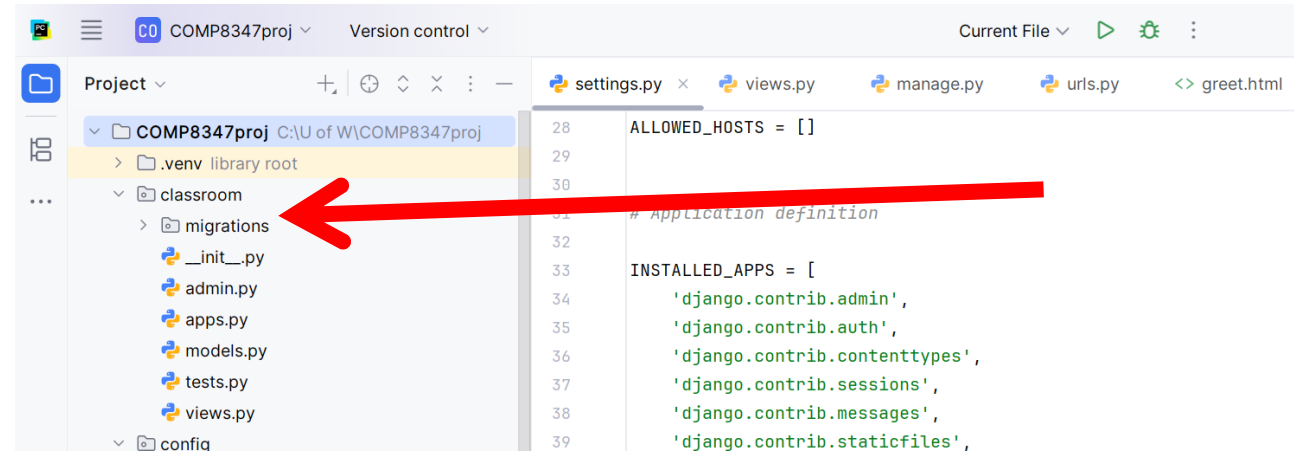
Open `config/settings.py` and add:

```
INSTALLED_APPS = [  
    # ...existing apps...  
    'classroom',  
]
```

Build the First Model

Open `classroom/models.py`:

```
from django.db import models  
  
class Student(models.Model):  
    name = models.CharField(max_length=100)  
    email = models.EmailField(unique=True)  
    join_date = models.DateField(auto_now_add=True)  
  
    def __str__(self):  
        return self.name
```

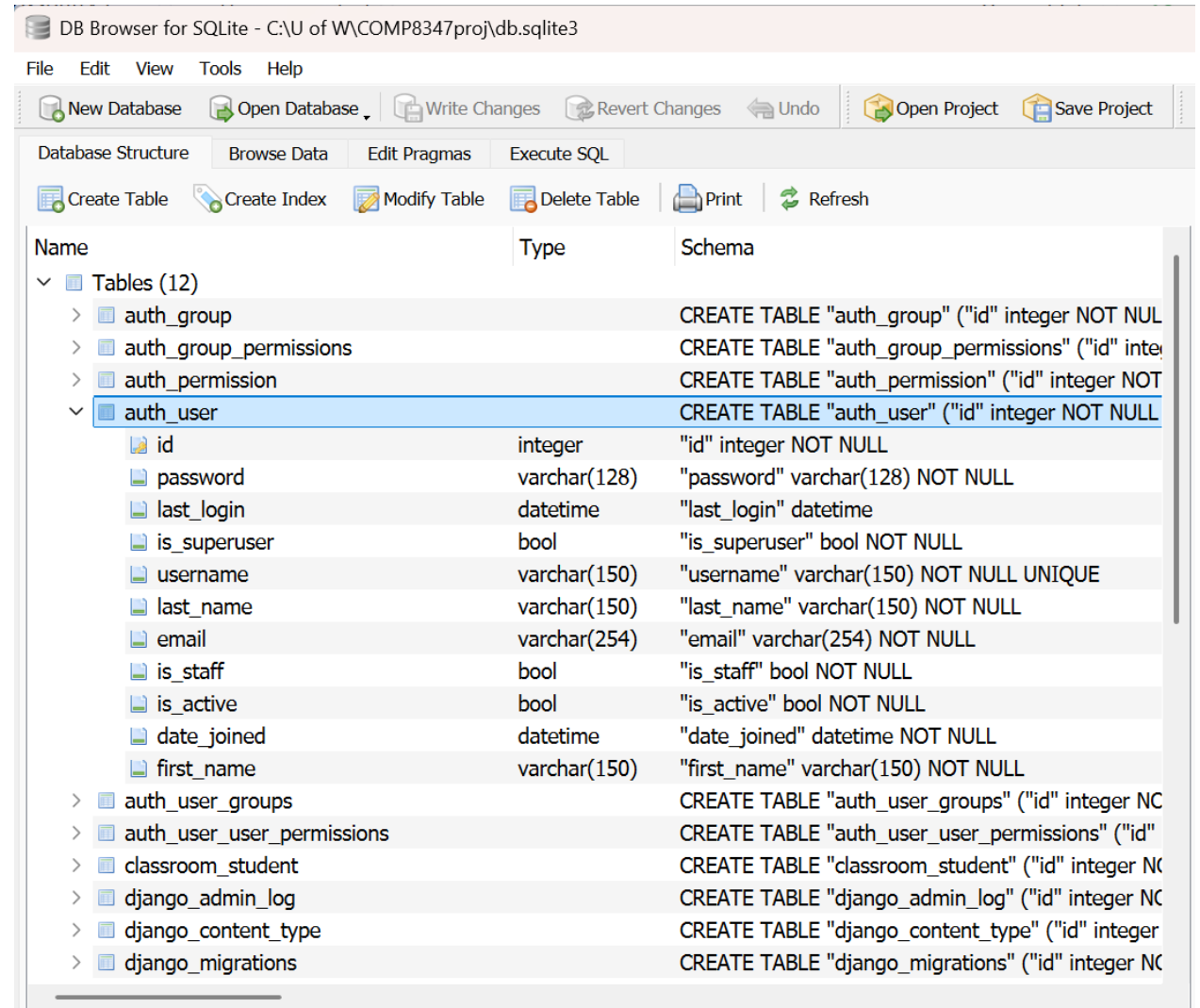


Migrations: Turning Code into a Table

In terminal:

```
python manage.py makemigrations
python manage.py migrate
```

- **makemigrations** scans for model changes and creates migration files.
- **migrate** applies those changes to the actual database (db.sqlite3 by default).
- Now we have two generated files:
db.sqlite3 and
migration file **0001_initial.py**



The screenshot shows the DB Browser for SQLite interface. The title bar indicates the database is located at C:\U of W\COMP8347proj\db.sqlite3. The menu bar includes File, Edit, View, Tools, and Help. The toolbar contains buttons for New Database, Open Database, Write Changes, Revert Changes, Undo, Open Project, and Save Project. Below the toolbar, there are tabs for Database Structure, Browse Data, Edit Pragma, and Execute SQL. The Database Structure tab is active, showing a tree view of the database schema. The tree is expanded to show the 'auth_user' table, which is highlighted. The table's columns and their data types are listed in the main pane.

Name	Type	Schema
Tables (12)		
auth_group		CREATE TABLE "auth_group" ("id" integer NOT NUL
auth_group_permissions		CREATE TABLE "auth_group_permissions" ("id" inte
auth_permission		CREATE TABLE "auth_permission" ("id" integer NOT
auth_user		CREATE TABLE "auth_user" ("id" integer NOT NULL
id	integer	"id" integer NOT NULL
password	varchar(128)	"password" varchar(128) NOT NULL
last_login	datetime	"last_login" datetime
is_superuser	bool	"is_superuser" bool NOT NULL
username	varchar(150)	"username" varchar(150) NOT NULL UNIQUE
last_name	varchar(150)	"last_name" varchar(150) NOT NULL
email	varchar(254)	"email" varchar(254) NOT NULL
is_staff	bool	"is_staff" bool NOT NULL
is_active	bool	"is_active" bool NOT NULL
date_joined	datetime	"date_joined" datetime NOT NULL
first_name	varchar(150)	"first_name" varchar(150) NOT NULL
auth_user_groups		CREATE TABLE "auth_user_groups" ("id" integer NC
auth_user_user_permissions		CREATE TABLE "auth_user_user_permissions" ("id"
classroom_student		CREATE TABLE "classroom_student" ("id" integer NC
django_admin_log		CREATE TABLE "django_admin_log" ("id" integer NC
django_content_type		CREATE TABLE "django_content_type" ("id" integer
django_migrations		CREATE TABLE "django_migrations" ("id" integer NC

Views & Templates with Data

View

Go to `classroom/views.py`

```
from django.shortcuts import render
from .models import Student

def student_list(request):
    qs = Student.objects.all()
    return render(request, "classroom/student_list.html", {
        "students": qs,
        "count": qs.count(),
        "is_empty": not qs.exists(),
    })
```

Views & Templates with Data

URL

Go to `config/urls.py`

```
from classroom.views import student_list

urlpatterns += [
    path("students/", student_list, name="student-list"),
]
```

Views & Templates with Data

Template

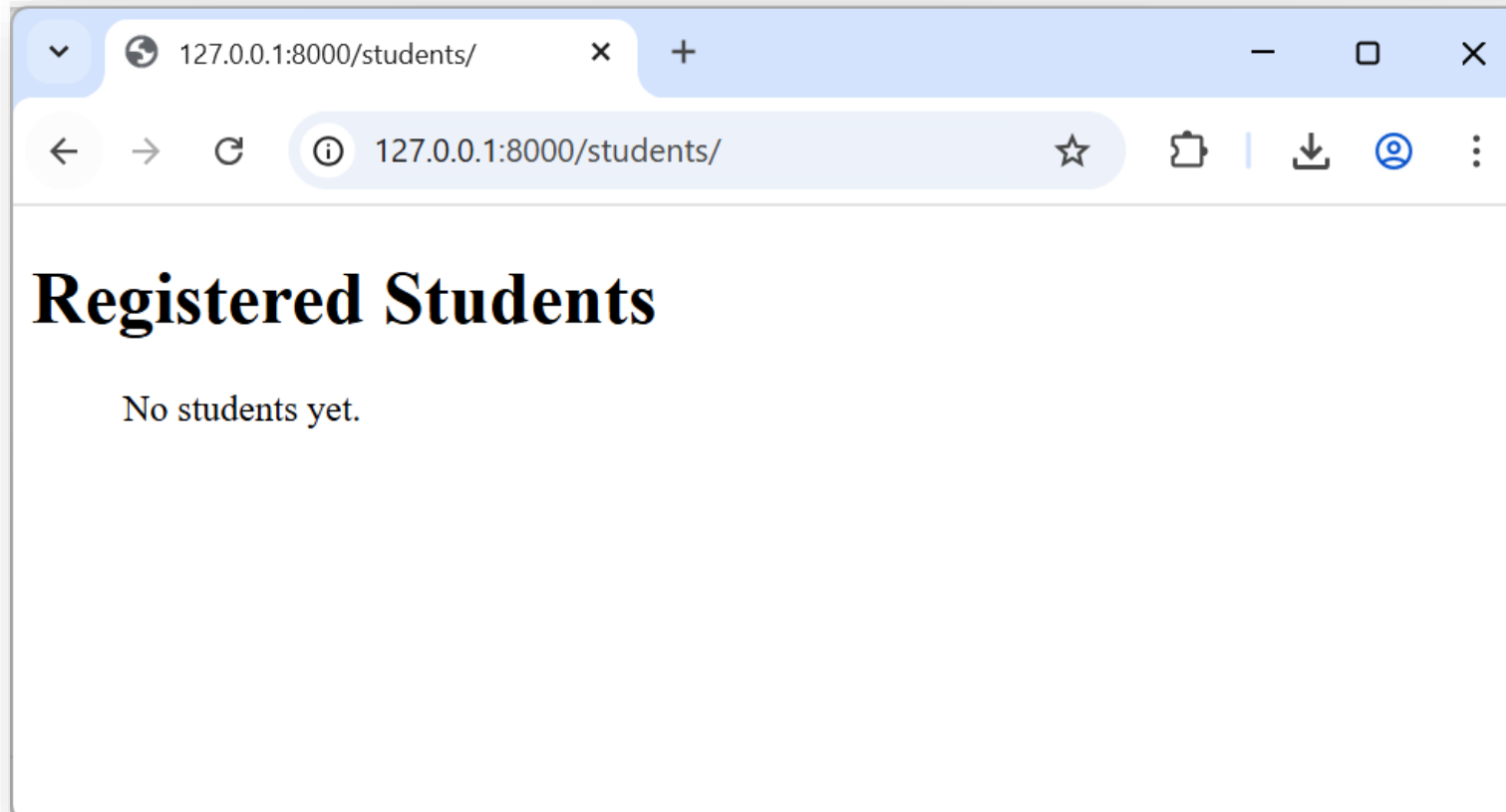
Go to `classroom/templates/classroom/student_list.html`

```
<!DOCTYPE html>
<html>
  <body>
    <h1>Registered Students</h1>
    <ul>
      {% if is_empty %}
        <p>No students yet.</p>
      {% else %}
        <ul>
          {% for s in students %}
            <li>{{ s.name }} - {{ s.email }} (joined {{ s.join_date }})</li>
          {% endfor %}
        </ul>
      {% endif %}
    </ul>
  </body>
</html>
```

Views & Templates with Data

Start the server and visit <http://127.0.0.1:8000/students/>

```
python manage.py runserver
```



Using the Django ORM

Instead of writing SQL e.g.

```
SELECT * FROM student WHERE name LIKE '%a%';
```

we can query with clean Python

Open the interactive shell and type:

```
python manage.py shell
```


Django QuerySet Methods

You will use those methods when retrieving data from your models:

1) `all()`

Purpose: Retrieve **every row** from the table.

`Student.objects.all()`

Returns: a **QuerySet** of all **Student** objects.

Note: Usually the starting point for building further filters.

Django QuerySet Methods

2) `filter(**lookups)`

Purpose: Get rows that **match specific conditions**.

Use keyword arguments where the key is a field name (with optional lookups such as **`__contains`**, **`__gte`**, etc.) and the value is what you're searching for.

```
# Students named Alice
```

```
Student.objects.filter(name="Alice")
```

```
# Students who joined after Jan 1, 2025
```

```
Student.objects.filter(join_date__gte="2025-01-01")
```

Returns: a **QuerySet** (can be empty or many rows).

Django QuerySet Methods

3) `get(**lookups)`

Purpose: Fetch exactly one object.

`Student.objects.get(id=5)`

Returns: a single **Student** instance.

- If **no match**, raises `Student.DoesNotExist`.
- If **more than one match**, raises `Student.MultipleObjectsReturned`.

Use only when you know exactly one row should match.

Django QuerySet Methods

4) `exclude(**lookups)`

Purpose: Opposite of `filter()` – get rows that **do NOT match** the condition.

```
# Everyone except Alice
```

```
Student.objects.exclude(name="Alice")
```

5) `order_by(*fields)`

Purpose: Sort the results.

```
# Ascending by name
```

```
Student.objects.order_by("name")
```

```
# Descending by join_date
```

```
Student.objects.order_by("-join_date")
```

Returns: a **QuerySet** ordered by the given field(s).

Django QuerySet Methods

4) Slicing (for limits and offsets)

QuerySets behave like Python lists, so you can use slicing for pagination or limiting results:

```
# First 5 students
```

```
Student.objects.all()[:5]
```

```
# Students 6 through 10
```

```
Student.objects.all()[5:10]
```

Django translates slicing into SQL LIMIT and OFFSET – it does **not** fetch all rows first.

Django QuerySet Methods

What does this code do?

```
# Latest 3 students named Alice  
Student.objects.filter(name="Alice") \  
    .order_by("-join_date")[:3]
```

Using the Django ORM

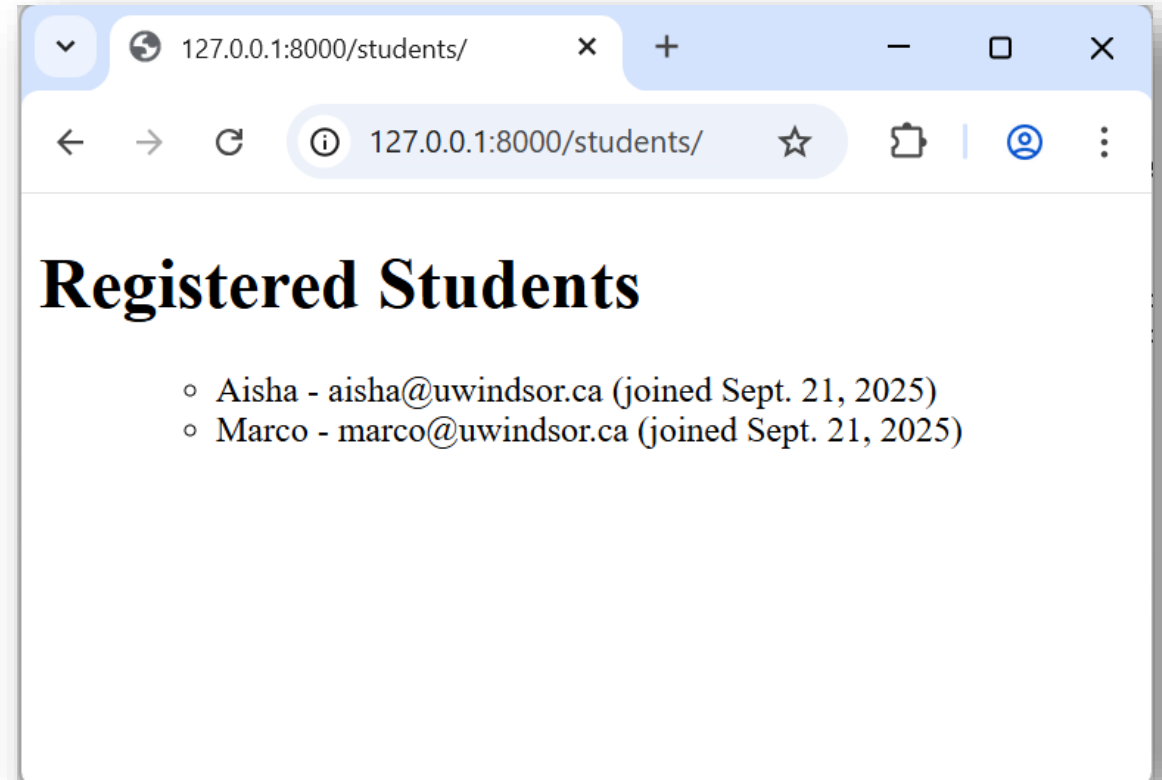
Open the interactive shell and type: **python manage.py shell**

```
from classroom.models import Student

# Create records
Student.objects.create(name="Aisha",
email="aisha@uwindsor.ca")

Student.objects.create(name="Marco",
email="marco@uwindsor.ca")

# Read all
Student.objects.all()
```



Using the Django ORM

Open the interactive shell and type: **python manage.py shell**

```
# Read all
```

```
Student.objects.all()
```

```
# Filter
```

```
Student.objects.filter(name__icontains="a")
```

```
# Order
```

```
Student.objects.order_by("-join_date")[:2]
```

```
# Update
```

```
m = Student.objects.get(email="marco@uwindsor.ca")
```

```
m.name = "Marcus"
```

```
m.save()
```

```
>>> Student.objects.all()
```

```
<QuerySet [<Student: Aisha>, <Student: Marco>]>
```

```
>>> Student.objects.filter(name__icontains="a")
```

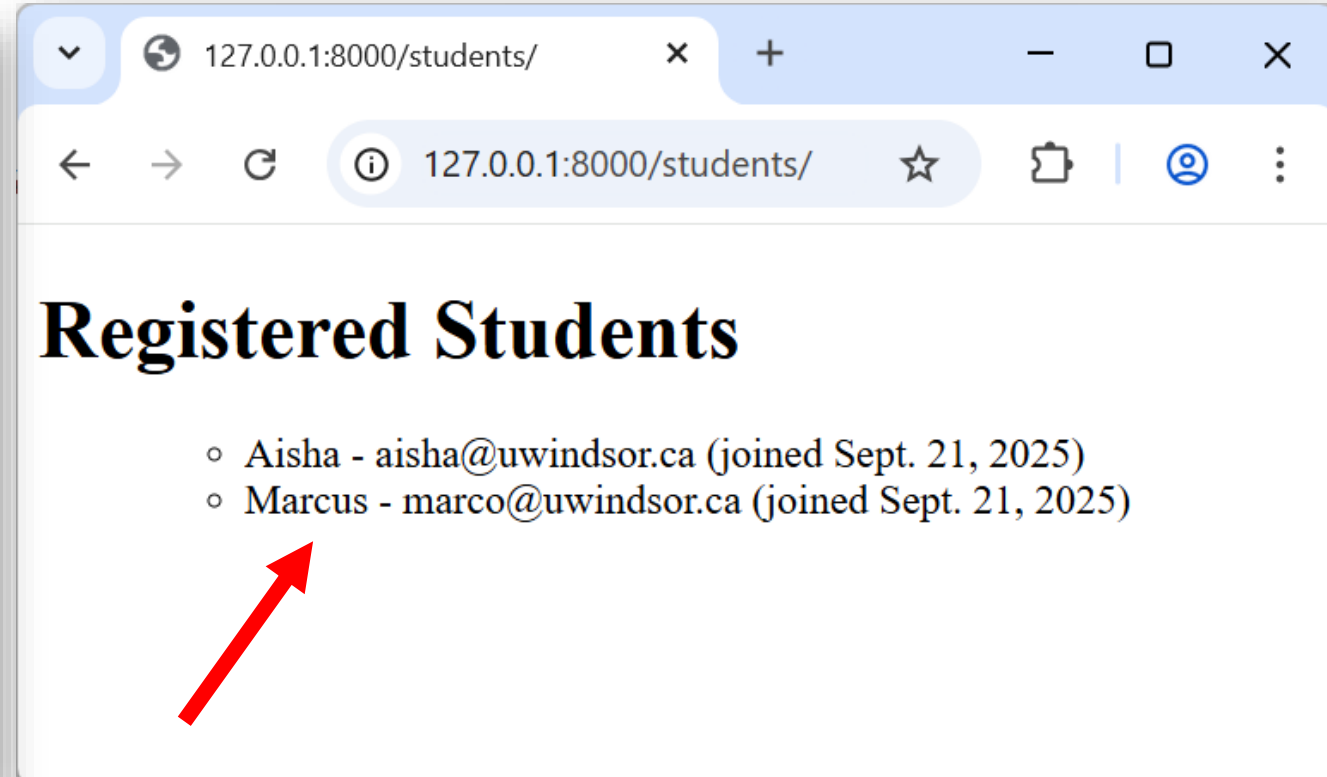
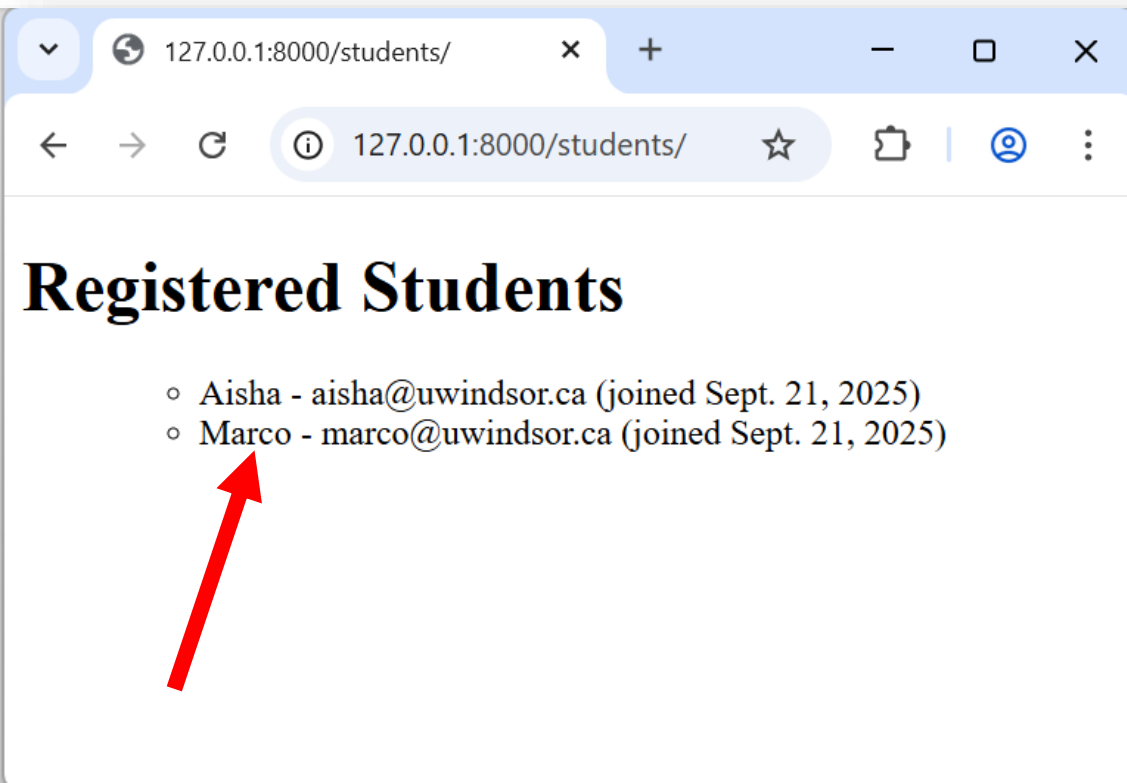
```
<QuerySet [<Student: Aisha>, <Student: Marco>]>
```

```
>>> Student.objects.order_by("-join_date")[:2]
```

```
<QuerySet [<Student: Aisha>, <Student: Marco>]>
```

Using the Django ORM

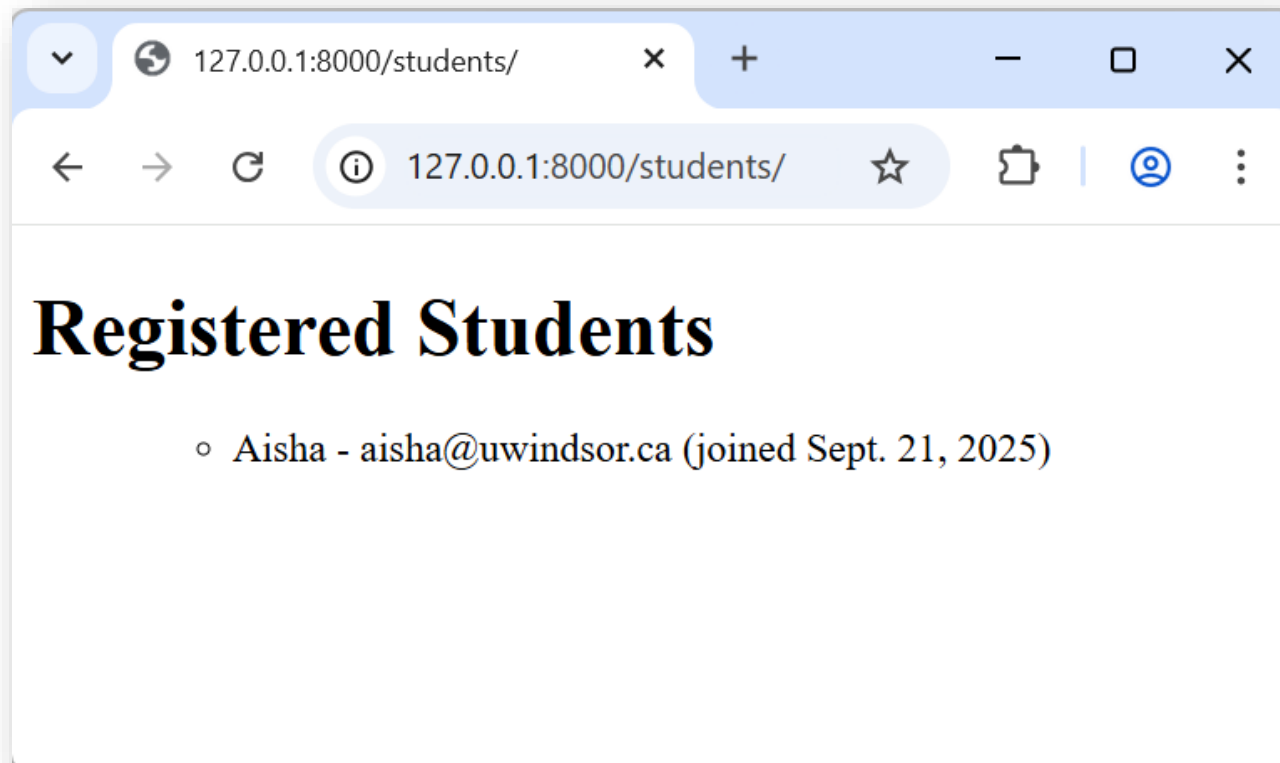
Open the interactive shell and type: `python manage.py shell`
After Update



Using the Django ORM

Open the interactive shell and type: `python manage.py shell`

```
# Delete  
m.delete()
```



Using the Django ORM (Full Script)

Open the interactive shell and type: **python manage.py shell**

```
from classroom.models import Student
# Create records
Student.objects.create(name="Aisha", email="aisha@uwindsor.ca")
Student.objects.create(name="Marco", email="marco@uwindsor.ca")
# Read all
Student.objects.all()
# Filter
Student.objects.filter(name__icontains="a")
# Order
Student.objects.order_by("-join_date")[:2]
# Update
m = Student.objects.get(email="marco@uwindsor.ca")
m.name = "Marcus"
m.save()
# Delete
m.delete()
```

Django Admin: Managing Data Without Code

Register the Model

Go to `classroom/admin.py`

```
from django.contrib import admin
from .models import Student
admin.site.register(Student)
```

Create an Admin User

```
python manage.py createsuperuser
```

Enter a username, email, and password.

Username: **SuperAdmin**

Email: **SuperAdmin@uwindsor.ca**

Password: **SuperPass123**

Django Admin: Managing Data Without Code

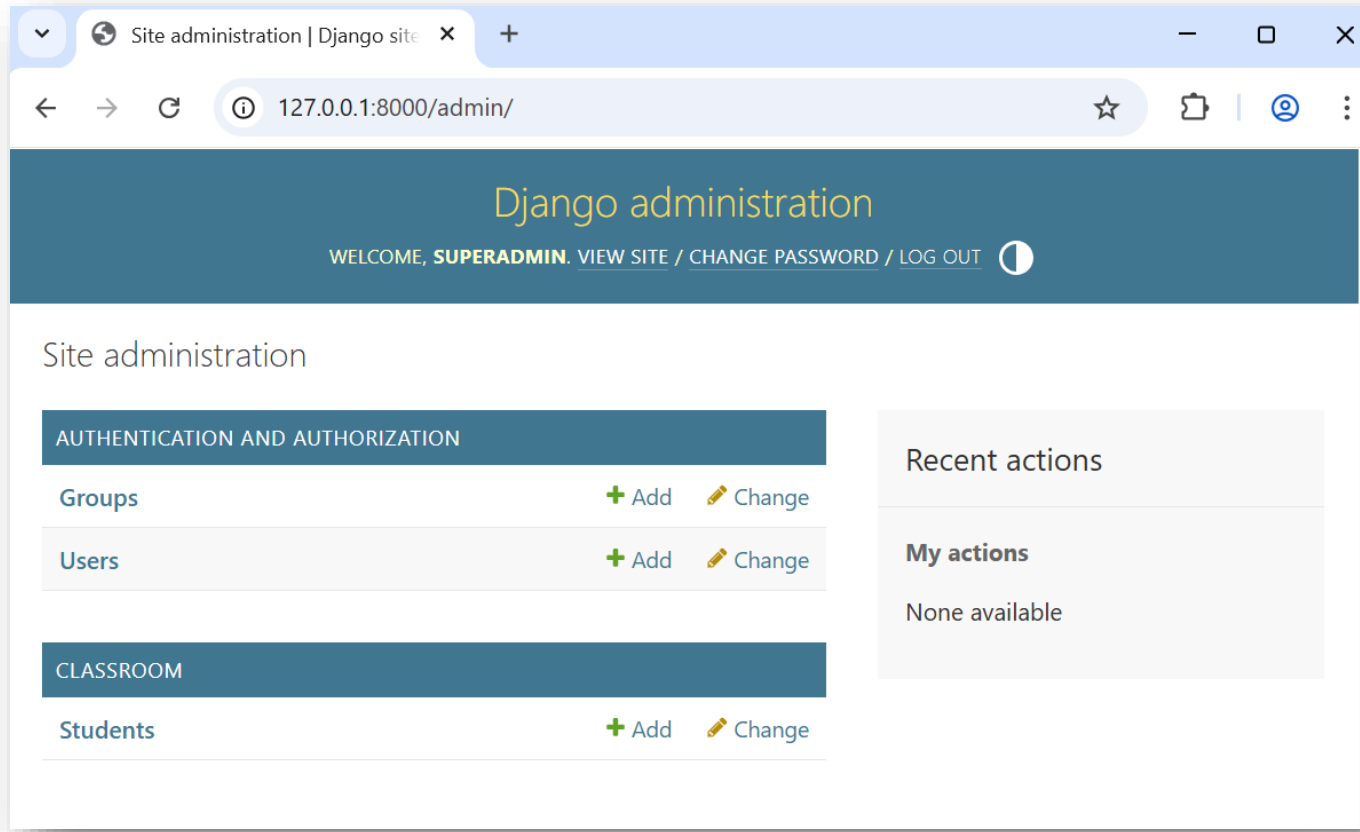
Username: **SuperAdmin**

Email: **SuperAdmin@uwindsor.ca**

Password: **SuperPass123**

```
python manage.py runserver
```

Log in at <http://127.0.0.1:8000/admin/> using the username and password you just set.



Django Admin: Managing Data Without Code

Username: **SuperAdmin**

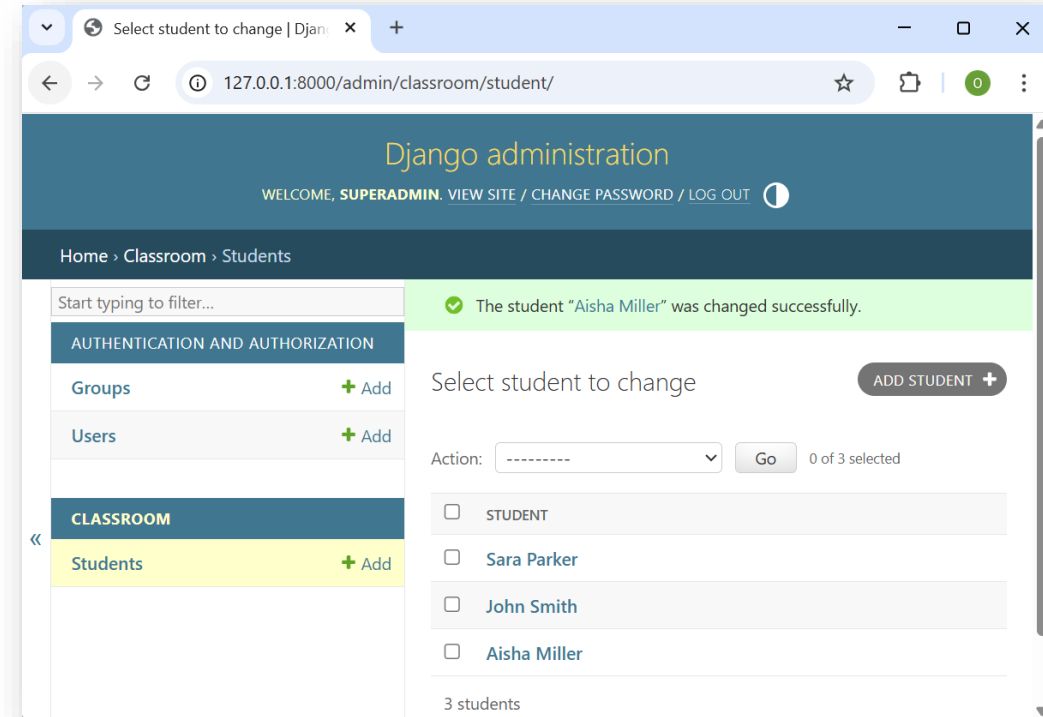
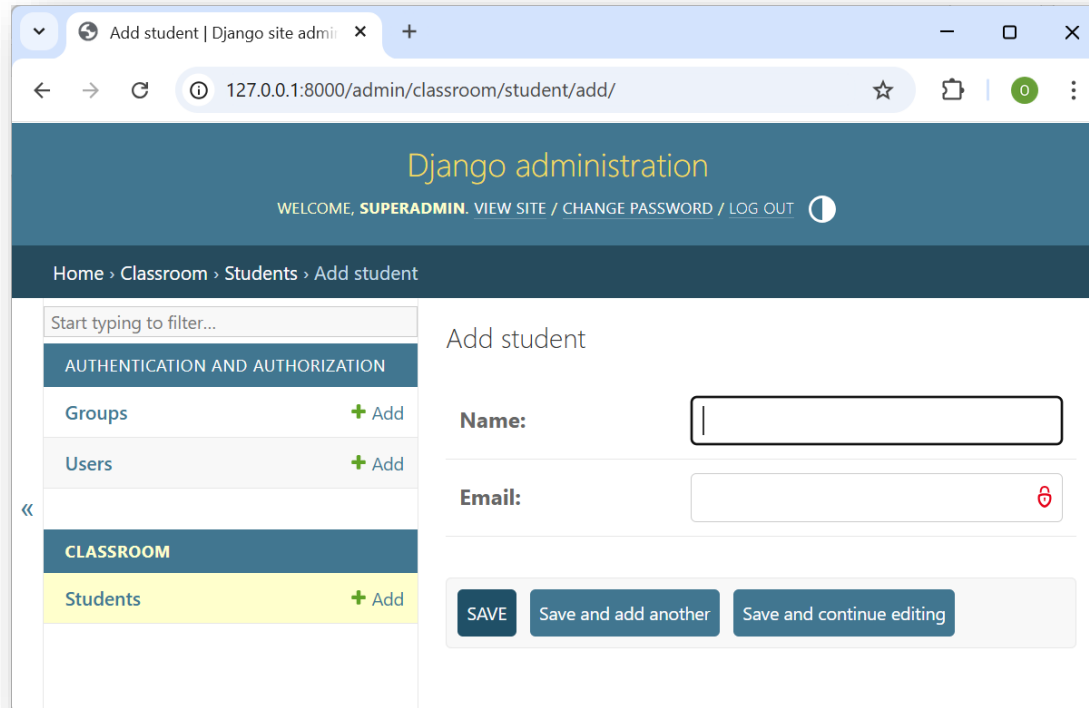
Email: **SuperAdmin@uwindsor.ca**

Password: **SuperPass123**

Add Student Records Through the Admin

- In the admin dashboard, click **Students** → **Add Student**.
- Fill in a name and email; join_date will auto-fill.
- Add at least three students.

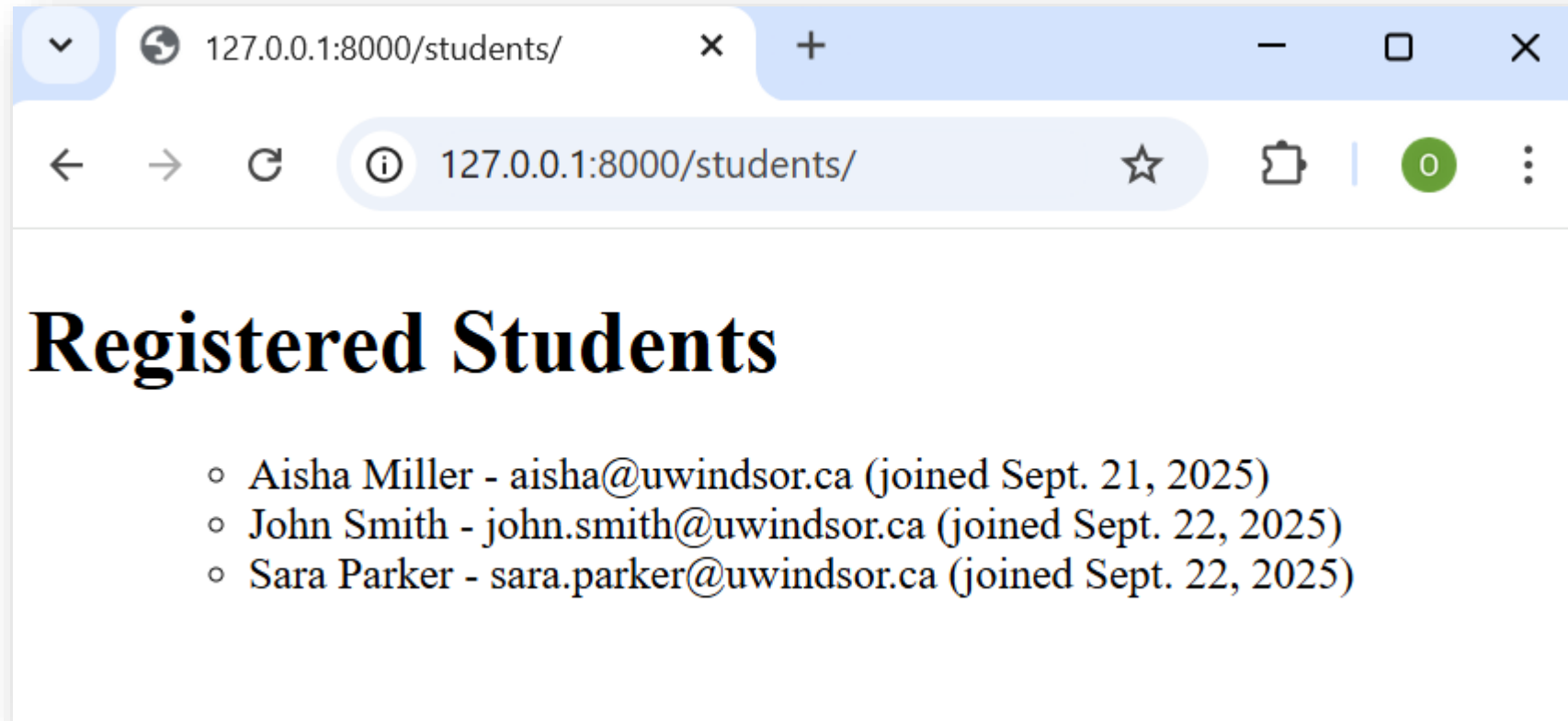
No code – yet our database now contains real records.



Django Admin: Managing Data Without Code

Verify in the Browser

Go to <http://127.0.0.1:8000/students/> (your existing student_list view).
You should now see the names you entered.



Adding CSS & Base Layout

Create a Base Template

Make a new file: **classroom/templates/classroom/base.html**

```
{% load static %}
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>{% block title %}Classroom{% endblock %}</title>
  <meta name="viewport" content="width=device-width,
initial-scale=1">
  <link rel="stylesheet" href="{% static
'classroom/css/site.css' %}">
</head>
<body>
  {% block content %}{% endblock %}
</body>
</html>
```

Adding CSS & Base Layout

Update Your Students Template

classroom/templates/classroom/student_list.html

```
{% extends "classroom/base.html" %}
{% block title %}Students{% endblock %}

{% block content %}
<style>
  body{margin:0;font-family:Arial,Helvetica,sans-serif;background:linear-
gradient(135deg,#a0d8ef,#f9f9f9);min-height:100vh}
  .page{background:#fff;max-width:700px;margin:40px auto;padding:28px;border-
radius:12px;box-shadow:0 8px 20px rgba(0,0,0,.1)}
  h1{margin:0 0 16px;color:#004c6d;text-align:center}
  table{width:100%;border-collapse:collapse;font-size:16px}
  th{background:#004c6d;color:#fff;text-align:left;padding:10px}
  td{padding:10px;border-bottom:1px solid #ddd}
  tr:nth-child(even){background:#f2f8fb}
  tr:hover{background:#e1f5ff}
  .empty{text-align:center;color:#777;padding:18px}
</style>
```

Adding CSS & Base Layout

Update Your Students Template

classroom/templates/classroom/student_list.html

```
<div class="page">
  <h1>Registered Students</h1>
  <table>
    <thead>
      <tr><th>Name</th><th>Email</th><th>Joined</th></tr>
    </thead>
    <tbody>
      {% for s in students %}
        <tr>
          <td>{{ s.name }}</td>
          <td>{{ s.email }}</td>
          <td>{{ s.join_date|date:"M d, Y" }}</td>
        </tr>
      {% empty %}
        <tr><td colspan="3" class="empty">No students yet.</td></tr>
      {% endfor %}
    </tbody>
  </table>
</div>
{% endblock %}
```

Adding CSS & Base Layout

Add Static Files

Create folder `classroom/static/classroom/css` and file `site.css`:

```
body {  
  margin: 0;  
  font-family: Arial, Helvetica, sans-serif;  
  background: linear-gradient(135deg, #a0d8ef, #f9f9f9);  
  min-height: 100vh;  
  display: flex;  
  justify-content: center;  
  align-items: flex-start;  
  padding-top: 40px;  
}  
  
.page {  
  background: white;  
  padding: 30px;  
  border-radius: 12px;  
  box-shadow: 0 8px 20px rgba(0,0,0,0.1);  
  width: 90%;  
  max-width: 700px;  
}
```

```
h1 {  
  text-align: center;  
  color: #004c6d;  
  margin-top: 0;  
  margin-bottom: 20px;  
}  
  
.student-table {  
  width: 100%;  
  border-collapse: collapse;  
  font-size: 16px;  
}
```

Adding CSS & Base Layout

Add Static Files

Create folder `classroom/static/classroom/css` and file `site.css`:

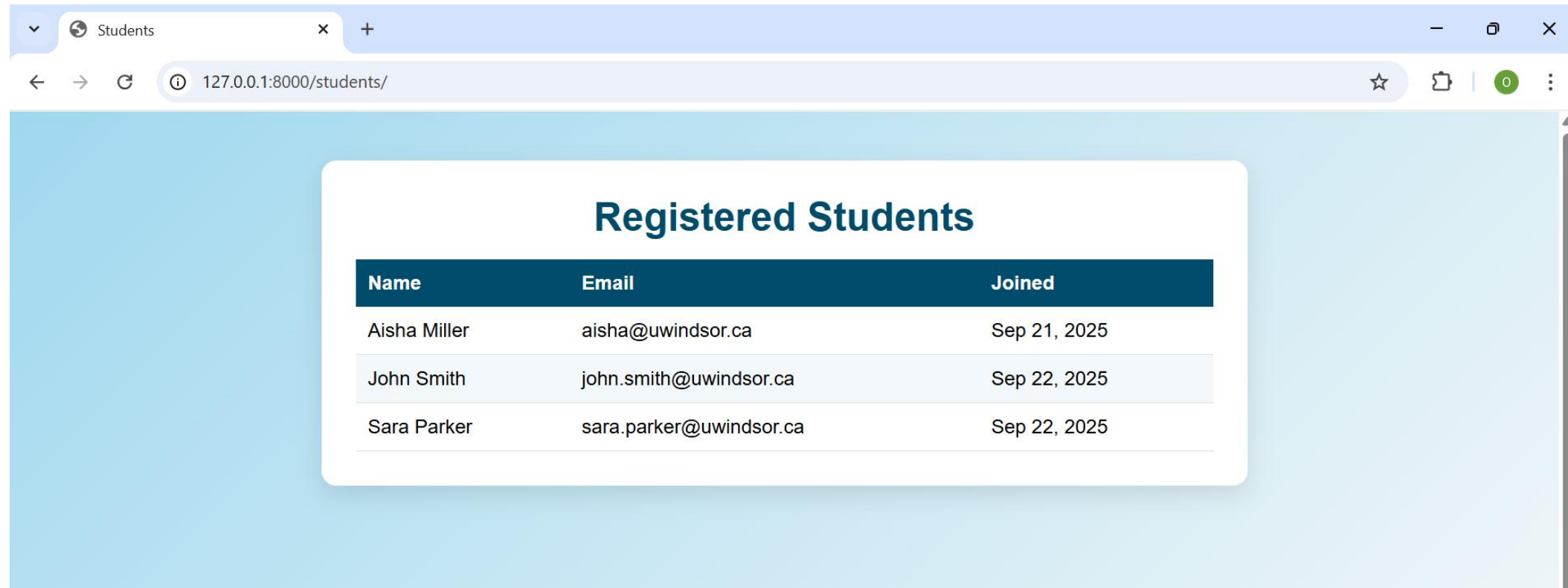
```
.student-table th {  
    background-color: #004c6d;  
    color: white;  
    padding: 10px;  
    text-align: left;  
}  
  
.student-table td {  
    border-bottom: 1px solid  
#ddd;  
    padding: 10px;  
}
```

```
.student-table tr:nth-child(even) {  
    background-color: #f2f8fb;  
}  
  
.student-table tr:hover {  
    background-color: #e1f5ff;  
}  
  
.empty {  
    text-align: center;  
    color: #888;  
    padding: 20px;  
}
```

Adding CSS & Base Layout

Refresh the Page

Visit <http://127.0.0.1:8000/students/> again. You should see the header, footer, and styled list.



A screenshot of a web browser window. The address bar shows '127.0.0.1:8000/students/'. The page has a light blue background. In the center, there is a white rounded rectangle containing the title 'Registered Students' in bold dark blue text. Below the title is a table with three columns: 'Name', 'Email', and 'Joined'. The table has three rows of student data. The first row is highlighted with a light blue background. The browser's tab bar shows 'Students' and the address bar has navigation icons and a star icon.

Name	Email	Joined
Aisha Miller	aisha@uwindsor.ca	Sep 21, 2025
John Smith	john.smith@uwindsor.ca	Sep 22, 2025
Sara Parker	sara.parker@uwindsor.ca	Sep 22, 2025

Class Activity – Challenge

- 1) Add a **phone_number** field to the Student model.
- 2) Create a page `/students/recent/` that lists students who joined in the last 7 days.
- 3) Write at least three queries in the shell (filter, order, delete) and submit them as a short report.